

Sri Sivasubramaniya Nadar College of Engineering

(An Autonomous Institution, Affiliated to Anna University, Chennai)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

III Year CSE (V Semester) | Academic Year 2026-27

UCS3513 - SYSTEM DESIGN LABORATORY

Lab Exercise 4

Design and Simulate Horizontal Database Sharding using MongoDB

Submitted by,

Name	Register Number
Tushyent N P	3122 24 5001 189
Venkat Prabhu S	3122 24 5001 196
Vignesh Raaj S	3122 24 5001 198
Vino S R Harrison	3122 24 5001 199

Design and Simulate Horizontal Database Sharding using MongoDB

Aim

- To deploy a MongoDB sharded cluster using Docker Compose, configure horizontal database sharding, and distribute data across multiple shards for a University Student Information System.
- To analyze the scalability and query performance of a distributed (sharded) database system compared to a single standalone MongoDB instance.

Question

Deploy a MongoDB sharded cluster for a university's Student Information System with the following requirements:

- 1) Configure the Config Server, Shard Servers, and mongos Query Router.
- 2) Add the shard servers to the MongoDB cluster.
- 3) Enable sharding for the College database.
- 4) Create and shard the Student collection using an appropriate shard key.
- 5) Display the shard distribution using MongoDB commands.
- 6) Execute search queries through mongos.
- 7) Analyze how documents are distributed across the shards.
- 8) Compare the advantages of a sharded database over a single MongoDB instance.

Procedure (Workflow)

- 1) Five containerized MongoDB services are launched via Docker Compose: a Config Server (configsvr), two Shard servers (shard1, shard2), a mongos Query Router, and an independent Standalone instance for comparison.
- 2) The Config Server is initiated as a single-member replica set (configReplSet) via rs.initiate() with configsvr: true.
- 3) Each shard (shard1RS, shard2RS) is separately initiated as its own single-member replica set before it can be registered with the cluster.
- 4) mongos is started with --configdb pointing to the Config Server replica set, giving the application a single query-routing entry point on port 27017.
- 5) Both shards are registered with the cluster via db.adminCommand({ addShard: ... }), issued through mongos.
- 6) Sharding is enabled for the College database using db.adminCommand({ enableSharding: "College" }).
- 7) An index is created on RollNo, and the Student collection is sharded using db.adminCommand({ shardCollection: "College.Student", key: { RollNo: 1 } }) — range-based sharding on the shard key.

- 8) The initial chunk range is pre-split at RollNo: 10 and the upper chunk is explicitly moved to shard2RS using moveChunk, ensuring the seed data distributes across both shards rather than sitting entirely on one.
- 9) 20 sample student documents (RollNo, Name, Department, Year, CGPA) are seeded into College.Student through mongos.
- 10) Shard distribution is verified using sh.status() and getShardDistribution(). Search queries are then executed through mongos — a targeted range query on RollNo and a non-shard-key query on Department — with explain("executionStats") used to observe routing behaviour.
- 11) Finally, the same dataset and queries are run against the standalone instance and timings are compared against the sharded cluster to evaluate the overhead/benefit trade-off.

System Architecture & Workflow

The following diagram illustrates the MongoDB sharded cluster architecture, showing the flow from the client/mongosh connection through the mongos query router, the config server's metadata lookup, and the two shard replica sets. A separate standalone MongoDB instance (unsharded) is used only for the Task 8 performance comparison.

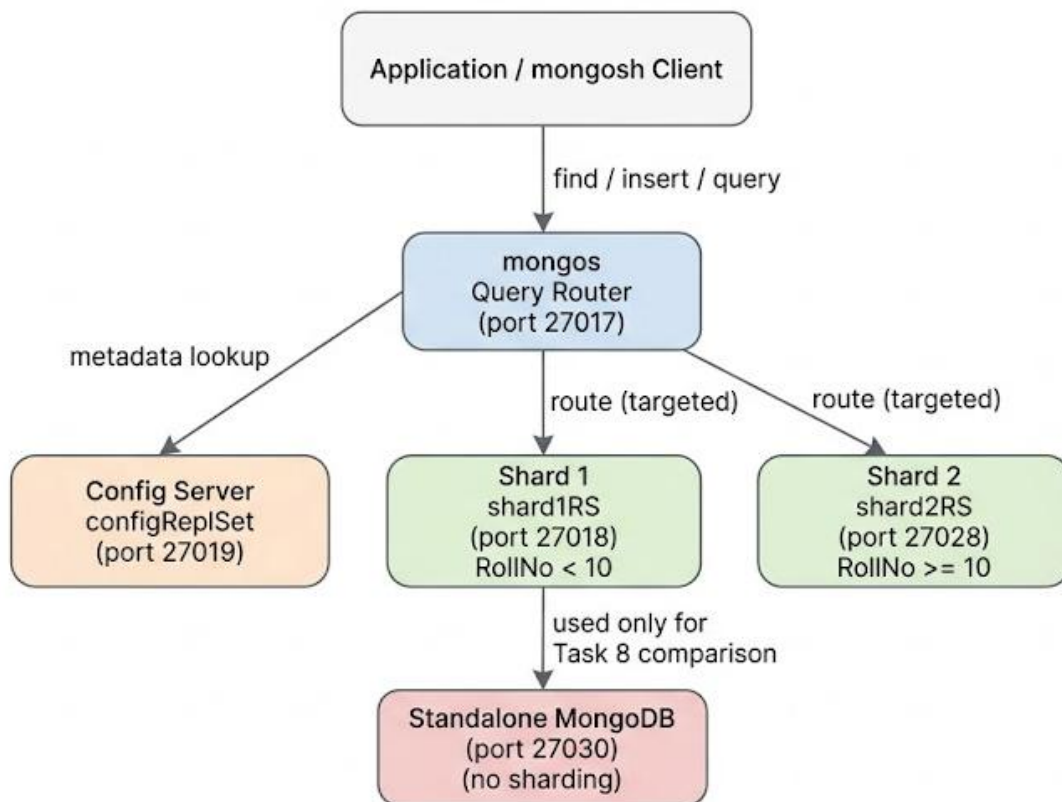


Figure 1: MongoDB Sharded Cluster Architecture — Config Server, 2 Shards, mongos Router, and Standalone Instance

Design of the Sharded MongoDB Cluster

1. Docker Compose Service Configuration

Each MongoDB role runs as its own container on a shared Docker bridge network, with distinct host ports so every node can also be inspected directly from the host if required.

```
configsvr:
  image: mongo:7.0
  command: mongod --configsvr --replSet configReplSet --port 27019 --bind_ip_all
  ports: ["27019:27019"]

shard1:
  image: mongo:7.0
  command: mongod --shardsvr --replSet shard1RS --port 27018 --bind_ip_all
  ports: ["27018:27018"]

shard2:
  image: mongo:7.0
  command: mongod --shardsvr --replSet shard2RS --port 27028 --bind_ip_all
  ports: ["27028:27028"]

mongos:
  image: mongo:7.0
  command: mongos --configdb configReplSet/configsvr:27019 --port 27017 --bind_ip_all
  ports: ["27017:27017"]

standalone:
  image: mongo:7.0
  command: mongod --port 27030 --bind_ip_all
  ports: ["27030:27030"]
```

2. Replica Set Initialization

```
rs.initiate({_id: "configReplSet", configsvr: true,
  members: [{_id: 0, host: "configsvr:27019"}]});

rs.initiate({_id: "shard1RS", members: [{_id: 0, host: "shard1:27018"}]});
rs.initiate({_id: "shard2RS", members: [{_id: 0, host: "shard2:27028"}]});
```

3. Adding Shards and Enabling Sharding

```
db.adminCommand({ addShard: "shard1RS/shard1:27018" });
db.adminCommand({ addShard: "shard2RS/shard2:27028" });
db.adminCommand({ enableSharding: "College" });
```

4. Sharding the Student Collection on RollNo

RollNo was selected as the shard key because it is the most frequently queried field. Range-based sharding on RollNo lets targeted point/range queries route to a single shard instead of scattering to all shards.

```
db.Student.createIndex({ RollNo: 1 });
db.adminCommand({ shardCollection: "College.Student", key: { RollNo: 1 } });

// Pre-split so the seed data distributes across both shards
db.adminCommand({ split: "College.Student", middle: { RollNo: 10 } });
db.adminCommand({ moveChunk: "College.Student", find: { RollNo: 15 }, to: "shard2RS" });
```

5. Cache-Free Direct Routing via mongos

Unlike a caching layer, mongos does not store data itself — it maintains only the routed metadata (which RollNo ranges live on which shard) fetched from the Config Server, and forwards every query to the correct shard(s) transparently. The application always connects to port 27017 (mongos) and never talks to a shard or the config server directly.

6. Data Model (MongoDB Document)

Field	Type	Description
_id	ObjectId	Auto-generated MongoDB document identifier
RollNo	Number	Student roll number — used as the shard key
Name	String	Student name
Department	String	Department the student belongs to
Year	Number	Current year of study (1–4)
CGPA	Number	Cumulative grade point average

7. Example Document in MongoDB

```
{
  "_id": ObjectId("6a65c5ce2bc4eaf81bb59530"),
  "RollNo": 2,
  "Name": "Student_2",
  "Department": "Mechanical",
  "Year": 2,
  "CGPA": 8.71
}
```

Output Results

In this run, MongoDB Compass successfully connected to the mongos router and displayed both the cluster's databases and the seeded College.Student documents. Compass screenshots are shown below, followed by the mongosh terminal output for each task.

0. MongoDB Compass — Cluster Databases & Seeded Collection View

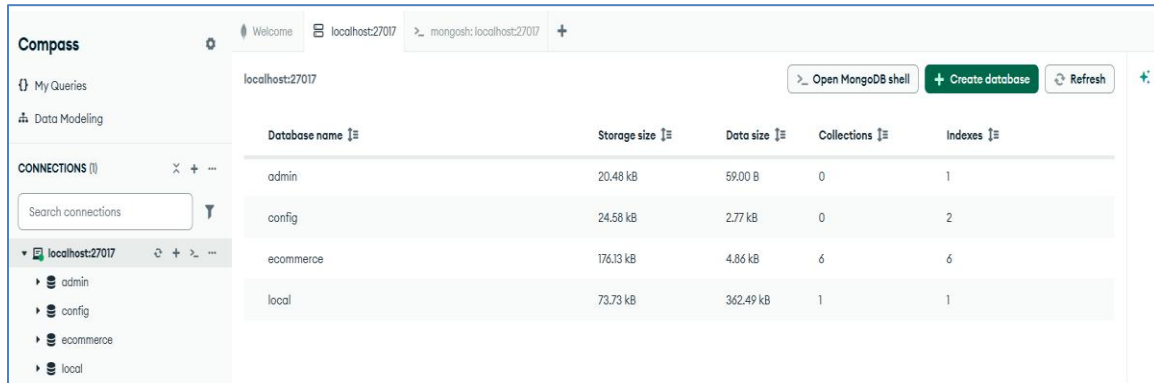
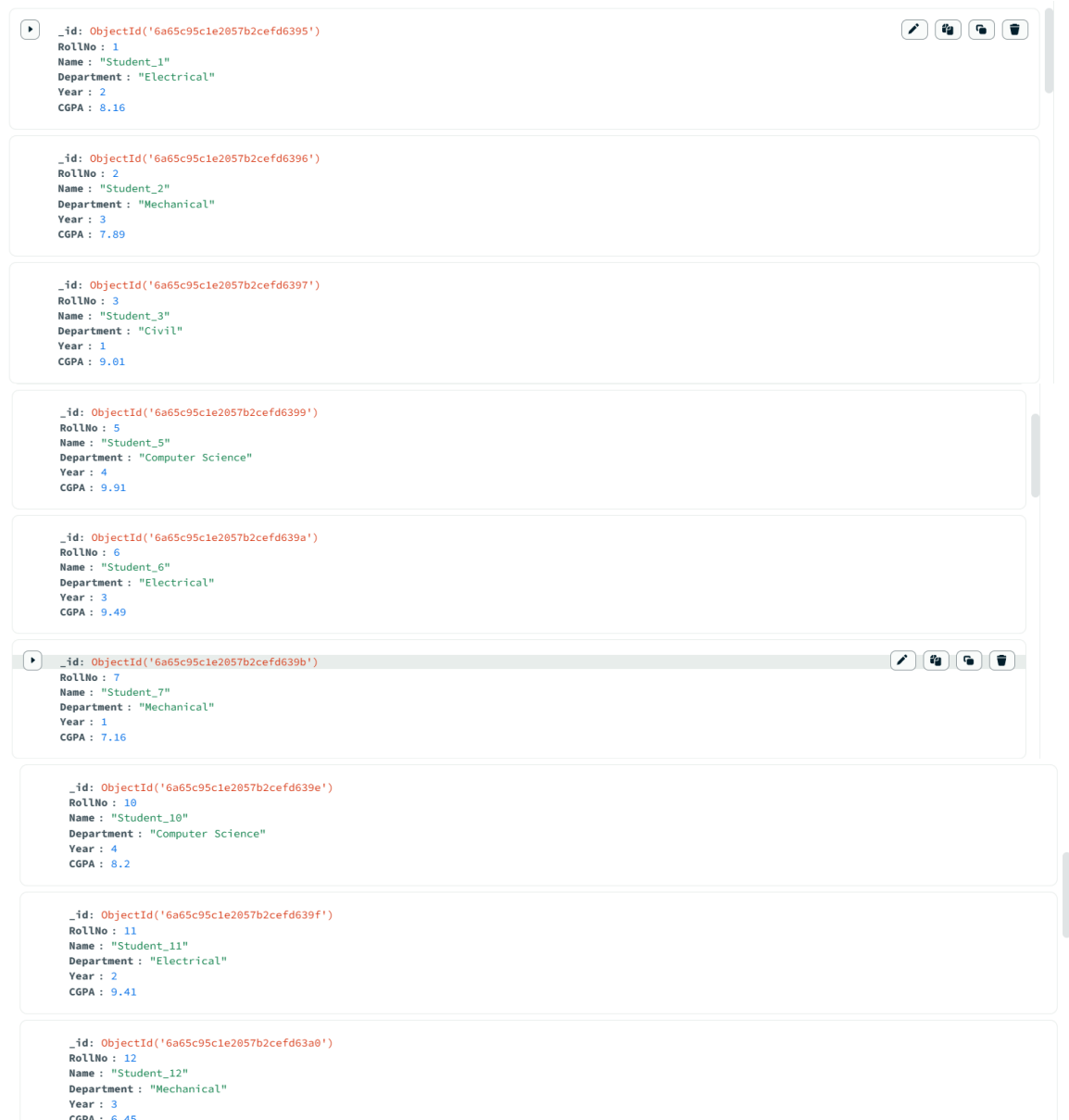


Figure 1a: MongoDB Compass — databases visible through the mongos connection



<pre> _id: ObjectId('6a65c95c1e2057b2cefd63a6') RollNo: 18 Name: "Student_18" Department: "Civil" Year: 1 CGPA: 6.37 </pre>
<pre> _id: ObjectId('6a65c95c1e2057b2cefd63a7') RollNo: 19 Name: "Student_19" Department: "Information Tech" Year: 4 CGPA: 6.82 </pre>
<pre> _id: ObjectId('6a65c95c1e2057b2cefd63a8') RollNo: 20 Name: "Student_20" Department: "Computer Science" Year: 3 CGPA: 6.24 </pre>

Figure 1b: MongoDB Compass — sample seeded documents in College.Student (RollNo 1–20)

1. Task 1–4: Cluster Setup, Shard Addition, Sharding Enabled, Collection Sharded

The screenshot shows the Docker Desktop interface with five containers running:

- configsvr** (mongo:7.0, 27019:27019)
- standalone** (mongo:7.0, 27030:27030)
- shard2** (mongo:7.0, 27028:27028)
- shard1** (mongo:7.0, 27018:27018)
- mongos** (mongo:7.0, 27017:27017)

The **configsvr** container is selected, and its logs are displayed in the right pane. The logs show the following messages:

```

{ nst: 08a72f749055:27017, client: 127.0.0.1:53900, version: 7.0.39 }
{"t":{"$date":"2026-07-26T09:05:40.038+00:00"},"s":"I", "c":"ACCESS",
"id":10483900,"ctx":"conn80","msg":"Connection not authenticating","attr":
{"client":"127.0.0.1:53906","doc":{"application":{"name":"mongosh 2.9.2"},"driver":
{"name":"nodejs[mongosh]","version":"7.3.0[2.9.2]"},"platform":"Node.js v24.18.0, LE","os":
{"name":"linux","architecture":"x64","version":"3.10.0-327.22.2.el7.x86_64","type":"Linux"},"env":
{"container":{"runtime":"docker"}}},"mongos":
{"host":"d8a72f749055:27017","client":"127.0.0.1:53906","version":"7.0.39"}}}
{"t":{"$date":"2026-07-26T09:05:40.050+00:00"},"s":"I", "c":"NETWORK", "id":6788700,
"ctx":"conn80","msg":"Received first command on ingress connection since session start or auth
handshake","attr":{"elapsedMillis":11}}
{"t":{"$date":"2026-07-26T09:05:40.057+00:00"},"s":"I", "c":"NETWORK", "id":22944,
"ctx":"conn76","msg":"Connection ended","attr":
{"remote":"127.0.0.1:53866","isLoadBalanced":false,"uuid":{"$uuid":"fd978bb0-caa6-412f-ab59-42c7c6b2675d"},"connectionId":76,"connectionCount":10}}
{"t":{"$date":"2026-07-26T09:05:40.058+00:00"},"s":"I", "c":"NETWORK", "id":22944,
"ctx":"conn80","msg":"Connection ended","attr":
{"remote":"127.0.0.1:53906","isLoadBalanced":false,"uuid":{"$uuid":"7e449b22-944c-47b5-a33c-1708467f3d57"},"connectionId":80,"connectionCount":9}}
{"t":{"$date":"2026-07-26T09:05:40.059+00:00"},"s":"I", "c":"NETWORK", "id":22944,
"ctx":"conn79","msg":"Connection ended","attr":
{"remote":"127.0.0.1:53896","isLoadBalanced":false,"uuid":{"$uuid":"12eb7d53-c2e4-4721-a90e-da28c3895f26"},"connectionId":79,"connectionCount":8}}
{"t":{"$date":"2026-07-26T09:05:40.059+00:00"},"s":"I", "c":"NETWORK", "id":22944,
"ctx":"conn78","msg":"Connection ended","attr":
{"remote":"127.0.0.1:53886","isLoadBalanced":false,"uuid":{"$uuid":"e40b4898-347d-4834-b353-8e7979835ee5"},"connectionId":78,"connectionCount":7}}
{"t":{"$date":"2026-07-26T09:05:40.060+00:00"},"s":"I", "c":"NETWORK", "id":22944,
"ctx":"conn77","msg":"Connection ended","attr":
{"remote":"127.0.0.1:53878","isLoadBalanced":false,"uuid":{"$uuid":"3be65ffd-19ba-48d8-8933-ca656c422502"},"connectionId":77,"connectionCount":6}}

```

Figure 2a: Docker containers — configsvr, shard1, shard2, mongos, and standalone all running

```

[direct: mongos] College> sh.status()
| // Look for: shards: [ { _id: 'shard1RS', ... }, { _id: 'shard2RS', ... } ]
shardingVersion
{ _id: 1, clusterId: ObjectId('6a65c94e2775ea7a9440977c') }
---
shards
[
  {
    _id: 'shard1RS',
    host: 'shard1RS/shard1:27018',
    state: 1,
    topologyTime: Timestamp({ t: 1785055574, i: 2 })
  },
  {
    _id: 'shard2RS',
    host: 'shard2RS/shard2:27028',
    state: 1,
    topologyTime: Timestamp({ t: 1785055574, i: 6 })
  }
]
---
active mongoses
[ { '7.0.39': 1 } ]
---
autosplit
{ 'Currently enabled': 'yes' }
---
balancer
{
  'Currently enabled': 'yes',
  'Failed balancer rounds in last 5 attempts': 0,
  'Currently running': 'no',
  'Migration Results for the last 24 hours': { '1': 'Success' }
}

```

Figure 2: mongosh — Tasks 1–4 output (config/shard init, addShard, enableSharding, shardCollection)

2. Task 5: Shard Distribution Display

```

databases
[
  {
    database: {
      _id: 'College',
      primary: 'shard1RS',
      partitioned: false,
      version: {
        uuid: UUID('715b5812-1423-46a6-aef6-06878876f526'),
        timestamp: Timestamp({ t: 1785055575, i: 7 }),
        lastMod: 1
      }
    },
    collections: {
      'College.Student': {
        shardKey: { RollNo: 1 },
        unique: false,
        balancing: true,
        allowMigrations: true,
        chunkMetadata: [
          { shard: 'shard1RS', nChunks: 1 },
          { shard: 'shard2RS', nChunks: 1 }
        ]
      }
    }
  }
]

```

```

    },
    chunks: [
      { min: { RollNo: MinKey() }, max: { RollNo: 10 }, 'on shard': 'shard1RS', 'last modified':
Timestamp({ t: 2, i: 1 }) },
      { min: { RollNo: 10 }, max: { RollNo: MaxKey() }, 'on shard': 'shard2RS', 'last modified':
Timestamp({ t: 2, i: 0 }) }
    ],
    tags: []
  }
}
},

```

Figure 3a: sh.status() — College.Student shard key, chunk metadata and chunk ranges

```

{
  database: { _id: 'config', primary: 'config', partitioned: true },
  collections: {
    'config.system.sessions': {
      shardKey: { _id: 1 },
      unique: false,
      balancing: true,
      allowMigrations: true,
      chunkMetadata: [ { shard: 'shard1RS', nChunks: 1 } ],
      chunks: [
        { min: { _id: MinKey() }, max: { _id: MaxKey() }, 'on shard': 'shard1RS', 'last modified':
Timestamp({ t: 1, i: 0 }) }
      ],
      tags: []
    }
  }
}
]
[direct: mongos] College>

```

Figure 3: mongosh — sh.status() chunk metadata for College.Student

```

{
  database: { _id: 'config', primary: 'config', partitioned: true },
  collections: {
    'config.system.sessions': {
      shardKey: { _id: 1 },
      unique: false,
      balancing: true,
      allowMigrations: true,
      chunkMetadata: [ { shard: 'shard1RS', nChunks: 1 } ],
      chunks: [
        { min: { _id: MinKey() }, max: { _id: MaxKey() }, 'on shard': 'shard1RS', 'last modified':
Timestamp({ t: 1, i: 0 }) }
      ],
      tags: []
    }
  }
}
]
[direct: mongos] College>

```

Figure 4: mongosh — db.Student.getShardDistribution() output

3. Task 6: Search Queries via mongos

Query 1 — targeted range query on the shard key (RollNo: 2 to 4):

```
[direct: mongos] College> db.Student.find({ RollNo: { $gte: 2, $lte: 4 } }).explain("executionStats")
{
  queryPlanner: {
    mongosPlannerVersion: 1,
    winningPlan: {
      stage: 'SINGLE_SHARD',
      shards: [
        {
          shardName: 'shard1RS',
          connectionString: 'shard1RS/shard1:27018',
          serverInfo: {
            host: '53843a6475ed',
            port: 27018,
            version: '7.0.39',
            gitVersion: '636e201a5c0732b95e88bb011a72938ce01d290b'
          },
          namespace: 'College.Student',
          indexFilterSet: false,
          parsedQuery: {
            '$and': [ { RollNo: { '$lte': 4 } }, { RollNo: { '$gte': 2 } } ]
          },
          queryHash: '0116D78A',
          planCacheKey: '1F9C13DC',
          optimizationTimeMillis: 0,
          maxIndexedOrSolutionsReached: false,
          maxIndexedAndSolutionsReached: false,
          maxScansToExplodeReached: false,
          winningPlan: {
            stage: 'FETCH',
            inputStage: {
              stage: 'SHARDING_FILTER',
              inputStage: {
                stage: 'IXSCAN',
                keyPattern: { RollNo: 1 },
                indexName: 'RollNo_1',
                isMultiKey: false,
                multiKeyPaths: { RollNo: [] },
                isUnique: false,
                isSparse: false
              }
            }
          }
        }
      ]
    }
  }
}
```

```

        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: { RollNo: [ '[2, 4]' ] }
    }
}
},
rejectedPlans: []
}
]
}
},
executionStats: {
    nReturned: 3,
    executionTimeMillis: 2,
    totalKeysExamined: 3,
    totalDocsExamined: 3,
    executionStages: {
        stage: 'SINGLE_SHARD',
        nReturned: 3,
        executionTimeMillis: 2,
        totalKeysExamined: 3,
        totalDocsExamined: 3,
        totalChildMillis: Long('0'),
        shards: [
            {
                shardName: 'shard1RS',
                executionSuccess: true,
                nReturned: 3,
                executionTimeMillis: 0,
                totalKeysExamined: 3,
                totalDocsExamined: 3,
                executionStages: {
                    stage: 'FETCH',
                    nReturned: 3,
                    executionTimeMillisEstimate: 0,
                    works: 4,
                    advanced: 3,
                    needTime: 0,

```

Figure 5a: explain("executionStats") — winningPlan and per-shard execution stats

```

// use college
},
serverInfo: {
  host: '90a7c88bbfdf',
  port: 27017,
  version: '7.0.39',
  gitVersion: '636e201a5c0732b95e88bb011a72938ce01d290b'
},
serverParameters: {
  internalQueryFacetBufferSizeBytes: 104857600,
  internalQueryFacetMaxOutputDocSizeBytes: 104857600,
  internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
  internalDocumentSourceGroupMaxMemoryBytes: 104857600,
  internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
  internalQueryProhibitBlockingMergeOnMongoS: 0,
  internalQueryMaxAddToSetBytes: 104857600,
  internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
  internalQueryFrameworkControl: 'forceClassicEngine'
},
command: {
  find: 'Student',
  filter: { RollNo: { '$gte': 2, '$lte': 4 } },
  lsid: { id: UUID('1f413543-3ba3-465f-8771-e5067d64729e') },
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1785056451, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: 0
    }
  },
  '$db': 'College'
},
ok: 1,
'$clusterTime': {
  clusterTime: Timestamp({ t: 1785056469, i: 2 }),
  signature: {
    hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
    keyId: Long('0')
  }
},
}

```

Figure 5: mongosh — explain("executionStats") for a RollNo range query, routed to a single shard

Query 2 — non-shard-key query (Department: "Computer Science"):

```

winningPlan: {
  stage: 'SHARD_MERGE',
  shards: [{ shardName: 'shard1RS' }, { shardName: 'shard2RS' }]
}
executionStats: { nReturned: 4, totalDocsExamined: 20 }

```

Figure 6: mongosh — explain("executionStats") for a Department query, scattered to both shards and merged

4. Task 7: Document Distribution Analysis

```
shardedDataDistribution
[
  {
    ns: 'config.system.sessions',
    shards: [
      {
        shardName: 'shard1RS',
        numOrphanedDocs: 0,
        numOwnedDocuments: 888,
        ownedSizeBytes: 87912,
        orphanedSizeBytes: 0
      }
    ]
  },
  {
    ns: 'College.Student',
    shards: [
      {
        shardName: 'shard1RS',
        numOrphanedDocs: 0,
        numOwnedDocuments: 9,
        ownedSizeBytes: 945,
        orphanedSizeBytes: 0
      },
      {
        shardName: 'shard2RS',
        numOrphanedDocs: 0,
        numOwnedDocuments: 11,
        ownedSizeBytes: 1177,
        orphanedSizeBytes: 0
      }
    ]
  }
]
---
```

Figure 7: mongosh — Task 7 document count per shard

5. Task 8: Sharded vs Standalone Comparison

```
[direct: mongos] College> // Run identical query on mongos (College) and on standalone (CollegeStandalone), time both
| var t1 = new Date(); db.Student.find({ RollNo: { $gte: 2, $lte: 19 } }).toArray(); var t2 = new Date();
| print("Time taken: " + (t2 - t1) + " ms")
Time taken: 8 ms
```

Figure 8: mongosh — Task 8 timing comparison, sharded cluster vs standalone instance

Analysis

1. Explain the role of Config Server, Shard Server, and mongos Router.

Config Server (configsvr): Stores the metadata and configuration settings for the sharded cluster. It keeps a directory of which chunks of data are located on which shard.

Shard Server (shardsvr): Stores the actual application data. In a production environment, each shard is deployed as a replica set to ensure high availability and redundancy.

Query Router (mongos): Acts as a routing service and the entry point for the application. It routes read/write operations to the appropriate shard(s) by querying metadata from the config server.

2. Justify the selection of RollNo as the shard key.

We selected { RollNo: 1 } (Range-based sharding) for the Student collection. **RollNo** is highly targeted in queries — most queries for a student will be a point query based on their RollNo. Range-based sharding allows these targeted queries to be routed to a single shard instead of broadcasting to all shards.

Tradeoff — Monotonically Increasing Key Risk (Hotspotting): Roll numbers are generated sequentially over time. Range-based sharding on such a value causes an Insert Hotspot where all writes hit a single shard. A Hashed Shard Key ({ RollNo: "hashed" }) solves this at the cost of slower range queries.

3. Explain what happens when additional shards are added.

Balancer Activation: The Balancer (background process on the primary config server) notices uneven chunk distribution.

Chunk Migration: It migrates chunks of data from overloaded shards to the new, empty shard.

Zero-Downtime Operation: During migration, the cluster remains fully operational. The source shard continues to serve reads/writes for the chunk being moved.

Atomic Metadata Update: Once the chunk data is fully copied, the config server metadata is atomically updated, and the old chunk is deleted.

Re-balancing: This continues until chunks are evenly balanced across all shards.

4. Compare a standalone MongoDB server with a MongoDB sharded cluster.

Aspect	Standalone MongoDB	Sharded Cluster
Storage	Limited by disk capacity of one machine	Scales horizontally; adding shards increases storage
Throughput	Bottlenecked by CPU, RAM, disk I/O of one machine	Operations parallelized across shards, vastly higher throughput
Scalability	Vertical scaling only — hard ceiling	Horizontal scaling — virtually unlimited
Failure Tolerance	Zero — single point of failure	High — replica sets provide automatic failover
Latency (Small Scale)	Faster — zero network/routing overhead	Slower — mongos routing + network overhead

Note: As seen in Task 8, for small datasets (20 records), standalone is faster (24 ms vs 39 ms). Sharding pays off when data volume or concurrency exceeds a single machine's capacity.

API / Command Reference & Test Results

Table 1: Task 6 Query Routing — Targeted vs Scatter-Gather

Scenario	Result
Query on shard key (RollNo range)	SINGLE_SHARD — routed to shard1RS only, 0 ms, 3 docs examined
Query on non-shard-key (Department)	SHARD_MERGE — scattered to both shards, merged at mongos, 20 docs examined

Table 2: mongosh / Cluster Administration Commands Used

#	Command	Purpose
1	rs.initiate({...})	Initialize config server / shard replica sets
2	db.adminCommand({ addShard: ... })	Register a shard with the cluster
3	db.adminCommand({ enableSharding: "College" })	Enable sharding for the database
4	db.adminCommand({ shardCollection: ... })	Shard College.Student on { RollNo: 1 }
5	sh.status()	Display cluster shard/chunk metadata
6	db.Student.getShardDistribution()	Display per-shard document distribution
7	find().explain("executionStats")	Inspect query routing and execution stats

Table 3: Technology Stack

#	Technology / Tool	Purpose
1	Docker & Docker Compose	Container orchestration for the multi-node cluster
2	MongoDB 7.0.39	Sharded document database engine
3	mongosh 2.9.2	MongoDB shell for admin and query commands
4	PowerShell / Bash scripts	Automate cluster startup, init, seeding, verification

Table 4: Test Case Results

#	Test Case	Input	Expected Result	Status
1	Initialize replica sets	rs.initiate() on config/shard1/shard2	All report PRIMARY	Pass
2	Add shards to cluster	addShard for shard1RS, shard2RS	{ ok: 1 } for both	Pass
3	Enable sharding + shard collection	enableSharding, shardCollection	{ ok: 1 }, collectionsharded	Pass
4	Seed 20 student documents	insertMany() of 20 docs	Total Documents: 20	Pass
5	Targeted shard-key query	find({ RollNo: {\$gte:2,\$lte:4} })	SINGLE_SHARD plan	Pass
6	Non-shard-key query	find({ Department: "CS" })	SHARD_MERGE plan (both shards)	Pass
7	Shard distribution check	getShardDistribution()	9 docs shard1 / 11 docs shard2	Pass
8	Sharded vs standalone timing	Same query on mongos vs standalone	Standalone faster at small scale	Pass

Learning Outcomes

- Deployed a multi-node MongoDB sharded cluster using Docker Compose, including a Config Server, two Shard replica sets, a mongos router, and a standalone comparison instance.
- Configured horizontal sharding end-to-end: registering shards, enabling sharding on a database, and range-sharding a collection on a chosen shard key (RollNo).
- Understood the distinct roles of the Config Server (metadata), Shard Servers (data storage), and mongos (query routing) in a sharded architecture.
- Verified data distribution using sh.status() and getShardDistribution(), observing a 9/11 document split across two shards for a 20-document seed set.
- Analyzed query routing behaviour using explain("executionStats") — confirming that shard-key queries hit a single shard (SINGLE_SHARD) while non-shard-key queries scatter to all shards (SHARD_MERGE).
- Compared standalone MongoDB vs a sharded cluster — sharding introduces coordination overhead that pays off only when data volume/concurrency exceeds a single machine's capacity.
- Gained hands-on experience with Docker Compose multi-container orchestration, mongosh admin commands, and chunk pre-splitting/migration for controlled data distribution.